Option	Description
--sample-book-mode	Same as --prompt simple.
--single-irb	Nested irb sessions will all share the same context.
--tracer	Displays trace for execution of commands.
-U	Same as Ruby's -U option, sets encoding to UTF-8.
-v, --version	Prints the version of irb.
--verbose, --noverbose	Do or don't show verbose details. The default is --noverbose
-W[level]	Set warning level, 0 is no warnings, 2 is verbose, one is in the middle.
-w	Suppress warning mode, like Ruby's -w.

Table 6—irb command-line options

Once started, irb displays a prompt and waits for you to type Ruby code. irb understands Ruby, so it knows when statements are incomplete. When this happens, irb will indicate that status as part of the prompt.

```
irb(main):001:0> 1 + 2
=> 3
irb(main):002:0* 3 +
irb(main):003:0> 4
=> 7
```

That * on line two happened after the + sign was typed, and indicates that the statement is incomplete. You get the same * if you type an open parenthesis. The first number between the colons is the line number, the second number between the colon and the final prompt is the indent level—you'll see if you add an open parenthesis that number will increase.

You can leave irb by typing exit or quit or by entering an end-of-file character (usually command-d). The latter behavior can be blocked by setting IGNORE_EOF mode. We'll talk about configuration options in the next section.

During an irb session, the work you do is accumulated in irb's workspace. Variables you set, methods you define, and classes you create are all remembered and may be used subsequently in that session.

```
irb(main):001:1* def fib_up_to(n)
irb(main):002:1*   f1, f2 = 1, 1
irb(main):003:2*   while f1 <= n
irb(main):004:2*     puts f1
irb(main):005:2*     f1, f2 = f2, f1 + f2
irb(main):006:1*   end
irb(main):007:0> end
=> :fib_up_to
irb(main):008:0> fib_up_to(4)
1
1
2
3
=> nil
irb(main):009:0>
```